

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT
TP. HỒ CHÍ MINH
Khoa Điện - Điện tử
Bộ môn Điện tử - Thông tin

Thực hành Học máy và Trí tuệ nhân tạo

Bài 8: Large Language Models for Natural Language Processing

Giảng viên: TS. Huỳnh Thế Thiện & PGS. TS Trương Ngọc Sơn

Tháng 03, 2026

1 Mục tiêu

Sau khi hoàn thành bài thực hành này, sinh viên có thể:

- Hiểu khái niệm Large Language Models (LLMs)
- Sử dụng pretrained language models
- Fine-tune mô hình BERT cho sentiment analysis
- Thực hiện text generation với GPT2
- Hiểu khái niệm Prompt Engineering
- Xây dựng chatbot đơn giản
- Hiểu cơ chế Retrieval-Augmented Generation (RAG)

2 Giới thiệu

Large Language Models (LLMs) là các mô hình deep learning được huấn luyện trên lượng dữ liệu văn bản rất lớn.

Một số LLM phổ biến:

- BERT
- GPT

- T5
- LLaMA

Các mô hình này có thể được sử dụng cho nhiều nhiệm vụ NLP:

- Text classification
- Question answering
- Text generation
- Translation
- Chatbot

Trong bài thực hành này, chúng ta sẽ sử dụng thư viện `transformers` của HuggingFace.

3 Chuẩn bị môi trường

Trên Kaggle hoặc Google Colab:

```
pip install transformers
pip install datasets
pip install sentence-transformers
pip install faiss-cpu
```

Import thư viện:

```
import torch
from datasets import load_dataset
from transformers import AutoTokenizer
from transformers import
    AutoModelForSequenceClassification
from transformers import TrainingArguments
from transformers import Trainer
from transformers import pipeline
```

4 Phần 1: Tải Dataset

Chúng ta sử dụng dataset IMDB cho sentiment analysis.

```
dataset = load_dataset("imdb")

print(dataset)
```

Hiển thị một ví dụ:

```
print(dataset["train"][0])
```

5 Phần 2: Tokenization

```
tokenizer = AutoTokenizer.from_pretrained(  
    "bert-base-uncased"  
)
```

```
def tokenize(example):  
  
    return tokenizer(  
        example["text"],  
        truncation=True,  
        padding="max_length"  
    )
```

```
tokenized_dataset = dataset.map(tokenize)
```

6 Phần 3: Fine-tuning BERT

Load model:

```
model = AutoModelForSequenceClassification.  
    from_pretrained(  
        "bert-base-uncased",  
        num_labels=2  
    )
```

Training arguments:

```
training_args = TrainingArguments(  
  
    output_dir="./results",  
  
    learning_rate=2e-5,  
  
    per_device_train_batch_size=8,  
  
    per_device_eval_batch_size=8,
```

```

        num_train_epochs=1,

        evaluation_strategy="epoch"
    )

```

Trainer:

```

trainer = Trainer(

    model=model,

    args=training_args,

    train_dataset=tokenized_dataset["train"].select(range
        (2000)),

    eval_dataset=tokenized_dataset["test"].select(range
        (1000))

)

```

Train:

```

trainer.train()

```

Evaluate:

```

trainer.evaluate()

```

7 Phần 4: Text Generation

Sử dụng GPT2.

```

generator = pipeline(
    "text-generation",
    model="gpt2"
)

result = generator(
    "Artificial intelligence will",

```

```

        max_length=40,

        num_return_sequences=2
    )

    print(result)

```

8 Phần 5: Prompt Engineering

Prompt ảnh hưởng rất lớn đến kết quả của LLM.

Ví dụ:

```

prompt = "Explain machine learning in simple terms:"

generator(prompt, max_length=60)

```

Thử các prompt khác nhau để quan sát sự thay đổi kết quả.

9 Phần 6: Mini Chatbot

Ta có thể tạo chatbot đơn giản:

```

chatbot = pipeline(
    "text-generation",
    model="gpt2"
)

def chat():

    while True:

        user_input = input("User: ")

        result = chatbot(
            user_input,
            max_length=60
        )

        print("Bot:", result[0]["generated_text"])

```

```
chat()
```

10 Phần 7: Retrieval Augmented Generation (RAG)

RAG kết hợp:

- Retrieval (tìm tài liệu liên quan)
- Generation (LLM sinh câu trả lời)

Ví dụ đơn giản:

```
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np

documents = [

    "Machine learning is a subset of AI",

    "Deep learning uses neural networks",

    "Transformers are used in NLP"

]

model = SentenceTransformer(
    "all-MiniLM-L6-v2"
)

embeddings = model.encode(documents)

index = faiss.IndexFlatL2(
    embeddings.shape[1]
)

index.add(np.array(embeddings))

query = "What is machine learning"

query_vector = model.encode([query])
```

```
D,I = index.search(
    np.array(query_vector),
    k=1
)

print(documents[I[0][0]])
```

11 Bài tập

1. Tải dataset IMDB.

Gợi ý:

- Sử dụng thư viện `datasets` của HuggingFace.
- Hàm `load_dataset` cho phép tải dataset trực tiếp từ HuggingFace Hub.
- Ví dụ:

```
from datasets import load_dataset

dataset = load_dataset("imdb")

print(dataset)
```

2. Hiển thị một số câu trong dataset.

Gợi ý:

- Truy cập phần `train` của dataset.
- In ra nội dung văn bản và nhãn sentiment.
- Ví dụ:

```
print(dataset["train"][0])

print(dataset["train"][1]["text"])
print(dataset["train"][1]["label"])
```

3. Tokenize dữ liệu bằng BERT tokenizer.

Gợi ý:

- Sử dụng `AutoTokenizer` từ thư viện `transformers`.

- Áp dụng tokenizer cho toàn bộ dataset bằng hàm `map()`.
- Ví dụ:

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained(
    "bert-base-uncased"
)

def tokenize(example):

    return tokenizer(
        example["text"],
        truncation=True,
        padding="max_length"
    )

tokenized_dataset = dataset.map(tokenize)
```

4. Fine-tune mô hình BERT trên một phần dataset.

Gợi ý:

- Sử dụng mô hình `AutoModelForSequenceClassification`.
- Sử dụng API `Trainer` của HuggingFace.
- Chỉ nên train trên một phần dataset để tiết kiệm thời gian.
- Ví dụ:

```
from transformers import Trainer
from transformers import TrainingArguments

training_args = TrainingArguments(

    output_dir="./results",

    num_train_epochs=1,

    per_device_train_batch_size=8
)
```

5. Tính accuracy trên tập test.

Gợi ý:

- Sử dụng hàm `trainer.evaluate()`.
- Có thể viết thêm hàm tính accuracy bằng cách so sánh:
 - nhãn dự đoán
 - nhãn thật
- Ví dụ:

```
import numpy as np

predictions = trainer.predict(
    tokenized_dataset["test"].select(range(500))
)

preds = np.argmax(predictions.predictions, axis=1)
labels = predictions.label_ids
```

6. Thử thay đổi learning rate.

Gợi ý:

- Thay đổi tham số `learning_rate` trong `TrainingArguments`.
- Ví dụ thử các giá trị:
 - `5e-5`
 - `2e-5`
 - `1e-5`
- So sánh accuracy sau khi huấn luyện.

7. Thực hiện text generation với GPT2.

Gợi ý:

- Sử dụng `pipeline("text-generation")`.
 - Ví dụ:
- ```
from transformers import pipeline

generator = pipeline(
 "text-generation",
 model="gpt2"
)

generator(
 "Artificial intelligence will",
```

```

 max_length=40
)

```

#### 8. Thử các prompt khác nhau.

Ví dụ:

- "The future of AI"
- "In the year 2050"
- "Machine learning is"

Gợi ý:

- Quan sát sự khác biệt của văn bản được sinh ra.
- Thử thay đổi tham số:
  - `max_length`
  - `num_return_sequences`
  - `temperature`

#### 9. Xây dựng chatbot đơn giản.

Gợi ý:

- Sử dụng vòng lặp `while` để nhận input từ người dùng.
- Gửi input này vào mô hình text generation.
- Ví dụ:

```

while True:

 text = input("User: ")

 result = generator(
 text,
 max_length=60
)

 print(result[0]["generated_text"])

```

#### 10. Thử nghiệm RAG với ít nhất 5 documents.

Gợi ý:

- Sử dụng `SentenceTransformer` để tạo embeddings.
- Lưu embeddings vào FAISS index.

- Khi có query mới, tìm document gần nhất.
- Ví dụ:

```
from sentence_transformers import
 SentenceTransformer
import faiss
import numpy as np
```

#### 11. Phân tích ưu điểm của RAG so với LLM thuần.

Gợi ý:

- RAG giúp mô hình truy cập thông tin từ dữ liệu bên ngoài.
- Giảm hiện tượng hallucination.
- Cho phép cập nhật tri thức mới mà không cần huấn luyện lại mô hình.